

Apache Spark Fundamentals

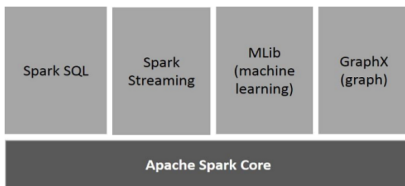
OUTLINE

- ① Introduction
- ② Spark Application
- ③ Resilient Distributed Datasets (RDD)
- ④ Implementing of MapReduce Algorithms in Spark

Introduction

- Spark is an open-source framework for developing and running applications on clusters. (Devised at UC Berkeley in 2009 and later donated to Apache Software Foundation).
- Spark provides
 - Unified computing engine (Spark Core)
 - Programming interface usable with Scala, Java, Python, R
 - APIs for data analysis: Spark SQL (structured data), MLlib (machine learning), GraphX (graph analytics), Spark Streaming (streaming analytics). Spark is written in Scala.

orchestrates the computation



- Spark runs on the Java Virtual Machine (JVM)

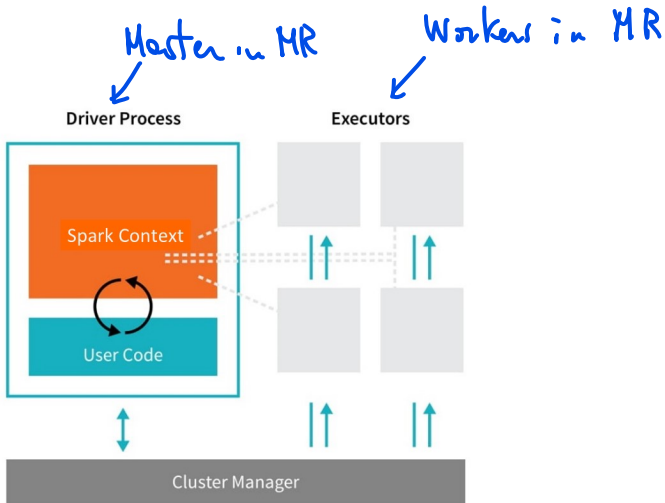
Introduction

- Spark does not come with a storage system (unlike Hadoop) but can run on the *Hadoop Distributed File System (HDFS)* as well as on other systems (e.g., HIVE or Relational DBMS).
- Spark's features:
 - Fault tolerance.
 - In-memory caching, which enables efficient execution of multiround algorithms, with a substantial performance improvement w.r.t. Hadoop.
- Spark can run:
 - On a single machine in the so called local mode. This is what we do in Homeworks 1 and 2.
 - On a cluster managed by a cluster manager such as Spark's Standalone, YARN, Mesos. For Homework 3 we will use the YARN cluster manager on CloudVeneto.

Why do we need a framework like Spark?

- * A single machine does not have enough power and resources for processing big data
⇒ need to pool power and resources of several machines into one platform (eg a cluster)
- * Spark: tool to manage and coordinate the execution of big data jobs on a cluster
Without such management and coordination the potential of the cluster is likely to be wasted

Spark Application



Spark Application

- **Driver process:** (*master process in MapReduce*) it is the heart of the application, and it
 - **Creates the Spark Context**, an object which can be regarded as a channel to access all Spark functionalities.
Obs.: Spark 2.0.0 introduced a **Spark Session object** which encapsulates the Spark Context and provides more functionalities.
 - **Distributes tasks to the executors.**
 - **Monitors the status of the execution.**

The driver runs Java/Python/Scala code through the Spark's API.

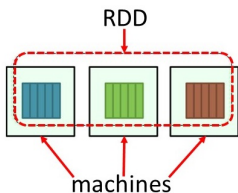
- **Executor processes:** (*worker processes in MapReduce*) they execute tasks assigned by the driver and report their status to the driver.
Executors run Scala code.

Spark Application

- **Cluster manager**: it controls physical machines and allocates *physical* resources to applications.
- **Driver and executors** are processes which can **run on the same machine or on different machines**. In **local mode**, both run (as threads) on one machine instead of a cluster.

Resilient Distributed Dataset (RDD)

- **Fundamental abstraction** in Spark. An RDD is a collection of elements of the same type, partitioned and distributed across several machines (if available).



- An RDD provides an interface based on **coarse-grained transformations**.
- RDDs ensure **fault-tolerance**.

RDD: Main characteristics

- RDDs are created
 - from data in stable storage (e.g., HDFS), or
 - from other RDDs, through transformations.
- RDDs are immutable (i.e., read-only).
- RDDs are materialized only when needed.
- Spark maintains the *lineage* of each RDD, namely the sequence of transformations that generate it, which enable to materialize it or reconstruct it after a failure, starting from data in stable storage.

RDD: Partitioning

- Key ingredient for efficiency: Each RDD is broken into chunks called partitions which are distributed among the available machines.
- A program can specify the number of partitions for each RDD (if not, Spark will choose one).
- Partitions are created by default (using a HashPartitioner, based on objects' hash codes) or through a custom partitioner.
- Typical number of partitions: is 2x/3x the number of cores, which helps *balancing the work*.
- Partitioning enables:
 - Parallelism. Some data transformations can be applied to the partitions in parallel, thus exploiting the parallelism offered by the underlying platform.
 - Data reuse. In multi-round applications, the same partitioning can be exploited in consecutive rounds, so each partition remains stored in the same machine and, possibly, in RAM.

RDD: Operations

The following types of operations can be performed on an RDD *A*

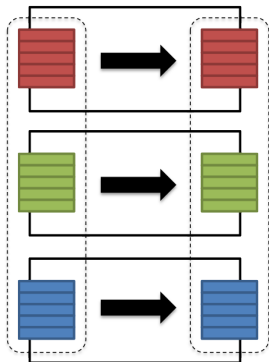
- **TRANSFORMATIONS.** A transformation generates a new RDD *B* starting from the data in *A*. We distinguish between:
 - **Narrow transformation.** Each partition of *A* contributes to (at most) one partition of *B*, which is stored in the same machine. No shuffling of data across machines is needed ($\Rightarrow$ maximum parallelism). E.g., the *map* method.
 - **Wide transformation.** Each partition of *A* may contribute to many partitions of *B*. Hence, shuffling of data across machines may be required. E.g., the *groupByKey* method.
- **ACTIONS.** An action is a computation on the elements of *A* which returns a value to the application. E.g., the *count* method.

IMPORTANT: It is at this point that *A* is materialized (*lazy evaluation*).

RDD: Operations

Narrow transformation

- Input and output stays in same partition
- No data movement is needed



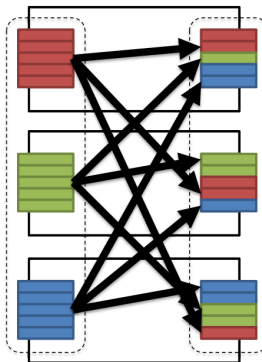
RDD A

RDD B

1 - 1

Wide transformation

- Input from other partitions are required
- Data shuffling is needed before processing



RDD A

RDD B

1 - many

Implementing MapReduce algorithms in Spark

The **homeworks** will provide you **first-hand experience** on how to **implement MapReduce algorithms** using Spark.

Summarized here are **key points to keep in mind** about this issue.

- Spark **enables the implementation of MapReduce algorithms** but offers a **much richer set of processing methods**.
- Spark (and MapReduce) exploits a **core idea of functional programming**: *functions can be argument to other functions*.
- A **program which implements a MapReduce algorithm** typically comprises
 - **Spark configuration steps** (e.g., creation of the context);
 - **Definitions of global variables and data structures**;
 - **Reading of input data** into one or more RDDs;
 - **Code for the various rounds**.

Implementing MapReduce algorithms in Spark

Typical Round

RDD X : input of the round (key-value pairs)

↓ Map Phase

RDD X' : intermediate data (key-value pairs)

↓ Reduce Phase

RDD Y : output of the round

X, X', Y are distinct RDDs but need not be materialized. They are materialized if some action on Y must be executed

Implementing MapReduce algorithms in Spark

- **Map Phase:** on the input RDD X invoke one of map methods offered by Spark (*narrow transformation*) passing the desired **map function** as argument.
- **Reduce Phase:** on the RDD X' resulting from the Map Phase:
 - invoke one of grouping methods offered by Spark (*wide transformation*) to group the key-value pairs into **key-ListOfValues** pairs;
 - invoke one of map methods offered by Spark to apply the desired **reduce function** to each key-ListOfValues pair.

The **reduceByKey** method allows you to do **both steps at once**.

Practical experience on Spark and on the implementation of MapReduce algorithms with Spark will be acquired through the homeworks.

Summary

- Spark features
- Spark Application: driver process, executor processors, cluster manager.
- Resilient Distributed Dataset (RDD)
 - Main characteristics
 - Partitioning
 - Operations: transformations, actions, persistence.
- Implementing MapReduce algorithms in Spark.

References

- AS-1 Spark's Web Site: `spark.apache.org`
- AS-2 Spark's RDD Programming guide:
`spark.apache.org/docs/latest/rdd-programming-guide.html`
- CZ18 A Gentle Introduction to Apache Spark. From B. Chambers, M. Zaharia. *Spark: The Definite Guide*, Databricks 2018.
- Z+12 M. Zaharia et al. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. NSDI 2012: 15-28.